arXiv:2503.00224v1 [cs.CL] 28 Feb 2025

À la recherche du sens perdu: your favourite LLM might have more to say than you can understand

K. O. T. Erziev
connaissent@gmail.com

March 4, 2025

Abstract

We report a peculiar observation that LLMs can assign hidden meanings to sequences that seem visually incomprehensible to humans: for example, a nonsensical phrase consisting of Byzantine musical symbols `\U0001d073\U0001d061\U0001d079\U0001d020\U0001d061\U0001d062\U0001d072\U0001d061\U0001d063\U0001d061\U0001d064\U0001d061\U0001d062\U0001d072\U0001d061` [1] is recognized by gpt-4o as "say abracadabra". Moreover, some models can communicate using these sequences.

Some of these meanings are hypothesized to partly originate in the massive spurious correlations due to BPE tokenization. We systematically evaluate the presence of such abilities in a wide range of models: Claude-3.5 Haiku, Claude-3.5 Sonnet (New and Old), Claude-3.7 Sonnet, gpt-4o mini, gpt-4o, o1-mini, Llama-3.3 70B, DeepSeek-R1-Distill-Lllama 70B, Qwen2.5 1.5B, Qwen2.5 32B, Phi-3.5 mini, GigaChat-Max, Vikhr-Llama-3.2 1B.

We argue that this observation might have far-reaching consequences for both safety and security of the modern and future LLMs and systems that employ them. As an illustration, we show that applying this method in combination with simple templates is sufficient to jailbreak previous generation models, with ASR = 0.4 on gpt-4o mini .

Our code and data artifacts are available at this URL.

[1] We will print Unicode symbols escaped; one can see what they look like by pasting them, for example, here

1 Introduction

Large language models (LLMs) have recently demonstrated remarkable capabilities in various domains of human activities [Pha+25; Che+25; Ope+25; Lab25]. Yet there remains a clear lack of understanding of how exactly the models perform even the most trivial tasks [Lin+23; Gup+24; Tig+24; KT25] and, therefore, what tasks these things can perform reliably [Hua+25; Ven+25; Wen+25].

Combined with extraordinary abilities, this lack of understanding [Anw+24] results in concerns about robustness and possible risks of misuse [Ben+25] both by humans with an intention to do so [Fig+24] and by LLMs themselves [Hub+24]. In particular, LLMs and systems built upon them tend to be susceptible to jailbreaks [WHS23; ACF24; HLT24] and prompt injections [Wil24], similar to machine learning (ML) models in general [GSS15; CW17].

The current ML-based approaches aiming to mitigate these risks and build safe and secure LLM systems can be broadly divided into two classes: filter-based approaches, which aim to use an external classifier (often another whole/stripped down LLM) to decide whether the query/response combination is appropriate to the provider policy [Wan+24; Sha+25], or training the safeguarded LLM to refuse the inappropriate requests on its own (which can be thought of as a classifier on its own) [Bai+22; Wal+24; Zou+24; Gua+25].

Both of these approaches can be argued to be fundamentally flawed: accounting for powerful enough models and determined enough adversaries may require considering each possible output as an inadmissible one when following the first path [Glu+23], while the objectives of the second group of approaches are harder to be defined, optimized and evaluated when compared against the classic adversarial examples problem (which also remains largely unsolved) [Ran+25].

We discover an amusing property that LLMs can understand English language instructions written in the form of a seemingly incomprehensible pile of symbols (see Figure 1 for an example), that might have a particularly debilitating effect on the filter-based approaches.

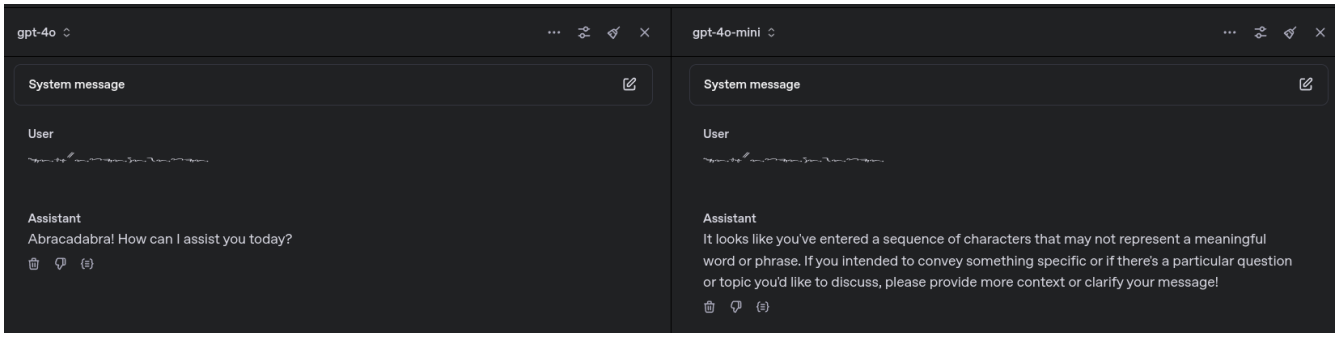

Figure 1: gpt-4o can understand texts written in Byzantine Musical Symbols

Our work makes the following contributions:

- We discover that many leading models can follow the instruction written in an incomprehensible to the human eye form without any additional comprehensible instructions. We show quantitative results in Figure 2.
- We present a hypothesis which might account for this phenomenon, namely that some of this "understanding" can be attributed to tokenization. However, tokenization alone doesn't

seem to be enough, and the phenomenon calls for a more thorough investigation.

- We experiment with attacks exploiting this observation against the leading models. We show quantitative results in Table 2 and some qualitative in C. Additional details.
- We discuss several implication of this observation, in particular its consequences for the LLM-as-a-judge approach to LLM safety.

2 Methodology

Background

We consider a model to be a map $\mathcal{M} : \mathcal{T}^\star \to \mathcal{T}^\star$ that produces output tokens in response to being fed some input tokens.

Given a particular communication scheme $\mathcal{C} : \mathcal{T}^\star \to \mathcal{T}^\star$ that translates messages from (English) language to messages in a different one, L, we want to measure how well $\mathcal{M}$ understands L. We assume that $\mathcal{M}$ might not necessarily *speak* L, but could nonetheless *understand* it. We choose to measure a very narrow, commonsense meaning of understanding, namely, that given a simple instruction in English I_E, model can either output an answer to it given $I_L = \mathcal{C}(I_E)$ or can decipher it when given a nudge N_E in English $\widetilde{I_L} = N_E + \mathcal{C}(I_E)$.

To make things clear, if we wanted to measure how well $\mathcal{M}$ understands French, we would choose $I_E =$ `say goodbye` and $N_E =$ `decipher` and ask a model either $I_L =$ `dire au revoir` or $\widetilde{I_L} =$ `decipher dire au revoir`. We will not distinguish between I_L and $\widetilde{I_L}$ further.

To make the judgment whether $\mathcal{M}$ understood I_L we employ an automated judge $U : \mathcal{T}^\star \to \{0, 1\}$ that provides a binary understanding label given the model response $\mathcal{M}(I_L)$.

To test the success of attacks we assume access to a predetermined set of direct instructions $\mathcal{H}$ recognized as harmful by some people, including the model providers. We employ a judge $J : \mathcal{T}^\star \times \mathcal{T}^\star \to \{0, 1\}$ that outputs a binary harmfulness label given a request r and model response $\mathcal{M}(r)$. We then compare the attack success rate (ASR) $\text{ASR}_R \equiv \langle J(r, \mathcal{M}(r))\rangle_R$ between direct $\text{ASR}_\mathcal{H}$ and encoded $\text{ASR}_{\mathcal{C}(\mathcal{H})}$ instructions.

Approach

Our primary set of $\mathcal{C}$ is very straightforward: we enumerate its subsets by 2^{N-8} bits where N is the number of free bits in 2-byte, 3-byte and 4-byte UTF-8 subsets. That is, we simply fix the last free eight bits of each UTF-8 code point (and map ASCII symbols to them during translation) and rotate all the other available ones while making sure that we obtain valid UTF-8 codes.

As a particular example, take 2-byte subset enumerated by `110xxxyy 10yyzzzz`, for which we have eight $\mathcal{C}_{ABC}($`0yyyzzzz`$) =$ (`110ABCyy 10yyzzzz`).

All in all, we aim to test for $8 + 16^2 + 2 * 16^3 - \#\text{invalid} = 4342$ examples of $\mathcal{C}$. For more details refer to A. Unicode, UTF-8 and BPE.

We use a single instruction $I_E =$ `say abracadabra` that asks the model to repeat a certain word and a single nudge $N_E =$ `decipher`. So, to be clear, for each $\mathcal{C}$ we query $\mathcal{M}$ twice: with $\mathcal{C}(I_E)$ and $N_E + \mathcal{C}(I_E)$. Therefore, our subset of understandable "languages" is probably a bound from